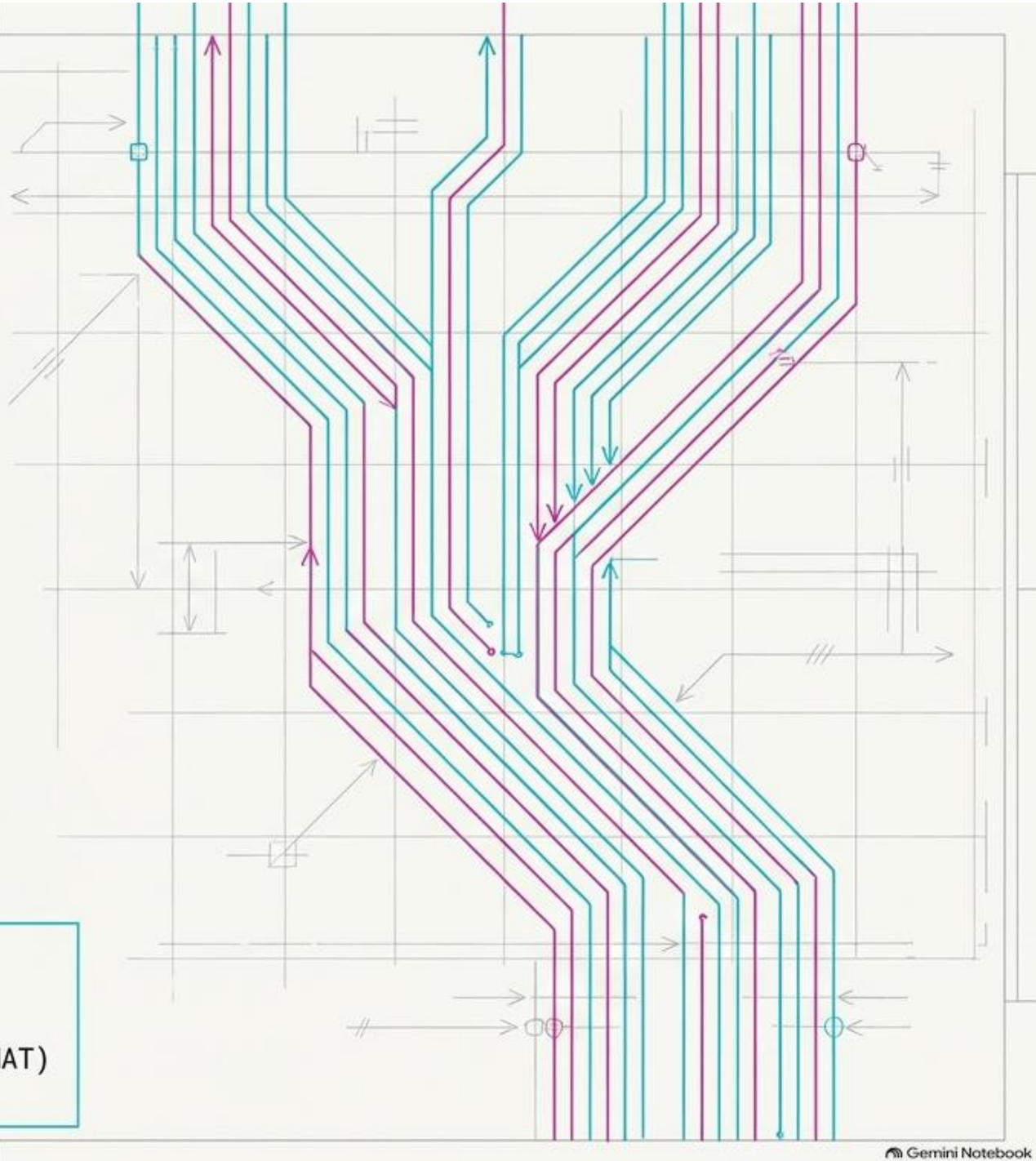
Threads: Fluxos de Execução e Multithreading

Uma análise da concorrência, compartilhamento e sincronização em Sistemas Operacionais



Professor: Prof. Me. Juvenal Neto
Disciplina: Sistemas Operacionais
Curso: Bacharelado em Ciência da Computação
Universidade: Universidade do Estado de Mato Grosso (UNEMAT)
Campus: Alto Araguaia

Gemini Notebook

Como um único processo pode realizar várias atividades ao mesmo tempo?

O Paradigma Clássico

Historicamente, um processo (um programa em execução) possui seu próprio espaço de endereçamento, código, dados e um PID (Identificador de Processo).

O Problema: Um processo clássico é single-thread. Ele possui apenas um fluxo de controle. Se ele trava esperando I/O, o processo inteiro para. Como contornar essa barreira sem o custo de criar um processo totalmente novo?



Gemini Notebook

O que é uma Thread?

Uma thread (ou processo leve) é a unidade básica de utilização da CPU. É um fluxo de execução independente que habita dentro do contexto de um processo maior.

Processo Single-thread (Antes)



Processo Multi-thread (Depois)



Insight: Um processo multithread gerencia tarefas simultâneas (ex: interface, renderização e rede) sem sair da fronteira de isolamento e proteção do processo original.

Gemini Notebook

O Mapa de Recursos: O que é Compartilhado vs. Privado?

Recursos Compartilhados (Processo / PCB)

- Seção de Código (Instruções)
- Seção de Dados (Variáveis Globais)
- Recursos do SO (Arquivos abertos, Sinais)

Contexto Privado (TCB)	Contexto Privado (TCB)	Contexto Privado (TCB)
- Contador de Programa (PC) - Registradores - Pilha / Stack	- Contador de Programa (PC) - Registradores - Pilha / Stack	- Contador de Programa (PC) - Registradores - Pilha / Stack

Atenção: Threads compartilham a ‘casa’ (Processo), mas cada uma necessita de sua própria ‘mesa de trabalho’ (Contexto Privado) para não misturar execuções.

Gemini Notebook

Matriz Diagnóstica: Processo vs. Thread

> Thread é um processo menor? (Não, é um fluxo dependente e leve).

Característica	Processo (Pesado)	Thread (Leve)
Custo de Criação / Troca	Alto / Lento (Aloca nova memória)	**Baixo / Rápido** (Reutiliza recursos do pai)
Espaço de Endereçamento	Isolado e Exclusivo	**Compartilhado** entre threads pares
Comunicação (IPC)	Complexa (Pipes, Sockets, SO)	**Simples** (Variáveis globais nativas)
Robustez e Segurança	Alta (Falha em um não afeta outro)	**Baixa** (Falha de memória derruba o processo)

Gemini Notebook

Concorrência vs. Paralelismo

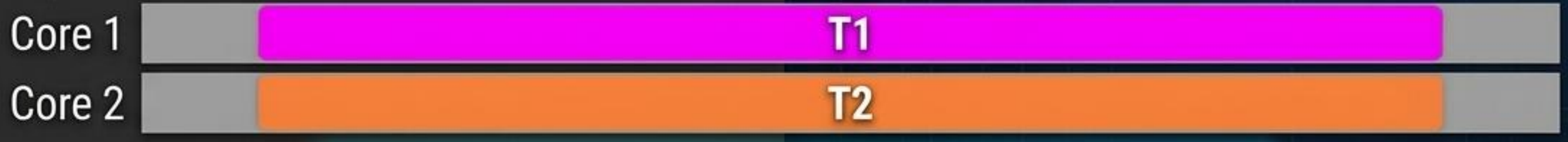
Duas threads sempre executam ao mesmo tempo físico?

Concorrência em 1 Core (Time-Slicing)



Alternância rápida de contexto cria a ilusão de simultaneidade.
Apenas uma executa no nível físico a cada instante.

Paralelismo em CPU Multicore



Recursos físicos independentes executam threads
literalmente no mesmo nanossegundo.

⚠ **Nota:** O multithreading é vital mesmo sem paralelismo! Ele impede que a aplicação inteira congele ao aguardar eventos de rede ou do usuário.

Gemini Notebook

Por Que Utilizar Multithreading?



1. Responsividade

A interface gráfica não congela. Ex: Um navegador rola a página fluidamente enquanto processa JavaScript pesado em background.



2. Compartilhamento Eficiente

Threads da mesma aplicação acessam o mesmo banco local. Ex: Servidor Web atende milhares de clientes sem replicar dados pesados.



3. Economia de Recursos

Criar e trocar contextos de threads custa uma fração minúscula de tempo de CPU comparado à chamada fork() tradicional.



4. Escalabilidade

Capacidade de explorar nativamente as arquiteturas modernas de hardware com dezenas de núcleos.

⚠ **Cuidado com a Lei de Amdahl:** Mais threads NÃO significam automaticamente maior desempenho. O limite é ditado pelo código não-paralelizável.

Gemini Notebook

Tornando o Invisível Visível: Observando Threads no Linux

```
$ ps -T -p 1234
  PID  SPID TTY          TIME CMD
 1234  1234 pts/0    00:00:02 myapp
 1234  1235 pts/0    00:00:01 myapp
 1234  1236 pts/0    00:00:00 myapp
```

SPID (TID): O ID único de cada thread. Note que o PID principal permanece idêntico para todas.

No Linux, threads são vistas como **LWP** (Lightweight Processes). O escalonador trata as LWPs individualmente.

```
$ top -H
Tasks: 362 total,   2 running, 360 sleeping,   0 stopped,   0 zombie
KiB Mem :  76393 total,   98471 free,  123845 task/rrceses
KiB Swap:      0 total,       0 uree,       0 soeechal fmod

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMD
 1234 root      20   0   13328  15326    366 S   0.0   0.2   0:00:02 myapp
 1235 root      20   0   12128  17129   1216 S   0.0   0.2   0:00:01 myapp
 1236 root      20   0   10338  10129   1265 S   0.0   0.1   0:00:01 myapp
 1237 root      20   0     530    520    217 S   0.0   0.1   0:00:00 myapp
 1238 root      20   0     435    323    318 S   0.0   0.1   0:00:00 myapp
 1239 root      20   0     360    410    322 S   0.0   0.1   0:00:00 myapp
```

```
$ ls /proc/1234/task/
1234   1235   1236   1237   1238   1239
```

O diretório virtual **/proc** revela subdiretórios individuais detalhando o estado de cada thread viva.

Gemini Notebook

Hands-on: Introdução ao POSIX Threads (pthreads)

```c
#include <pthread.h>
#include <stdio.h>

void *runner(void *param);  /* Tarefa assíncrona */

int main() {
    pthread_t tid; /* O 'RG' da nova thread */
    pthread_create(&tid, NULL, runner, NULL);
    pthread_join(tid, NULL); /* Barreira de sincronização */
    return 0;
}
```

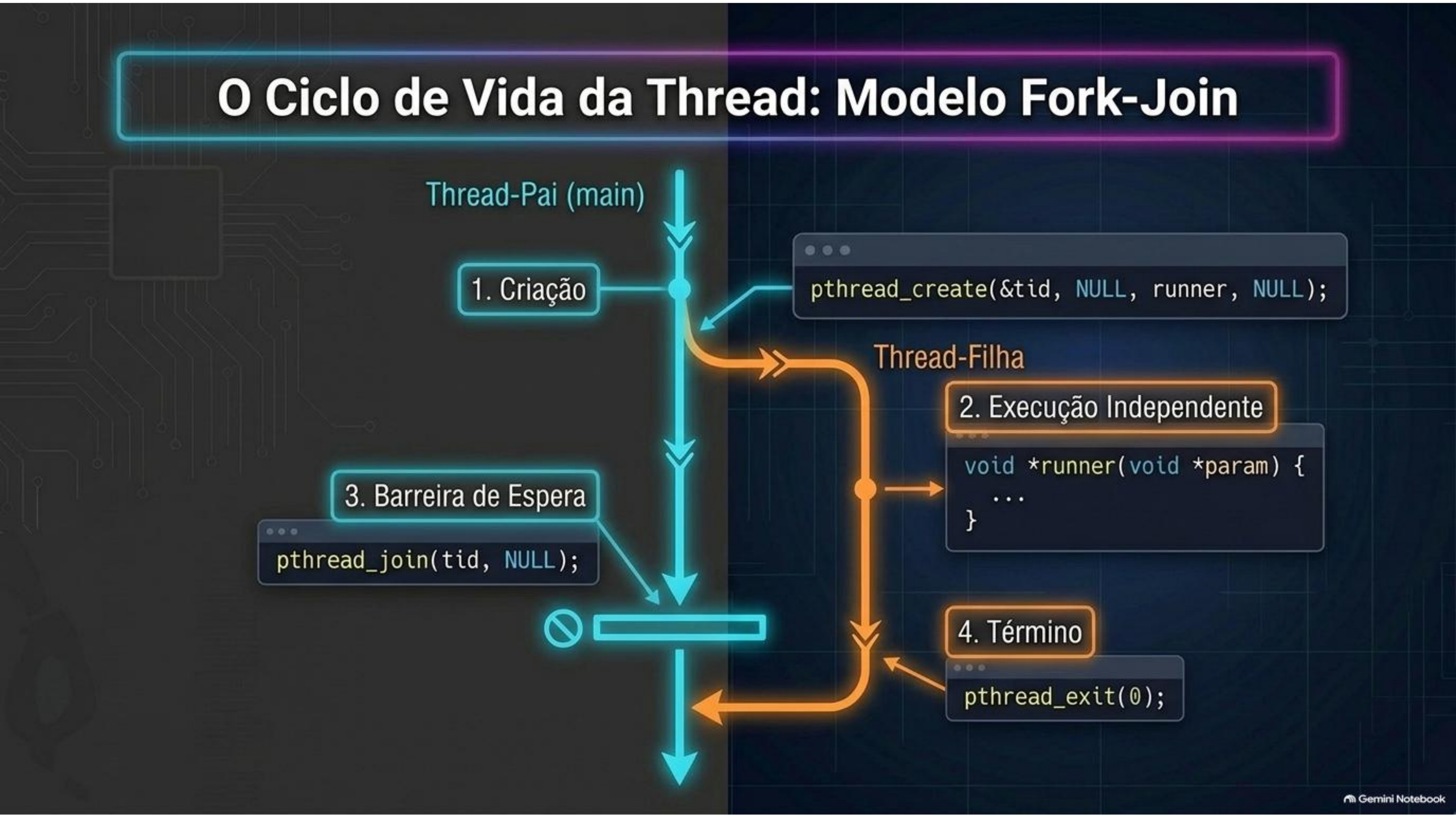
Declaração do identificador único da thread e seu TCB.

```
$ gcc programa.c -o programa -pthread
```

Flag obrigatória de compilação para realizar o link correto com a biblioteca POSIX no Linux.

Gemini Notebook

O Ciclo de Vida da Thread: Modelo Fork-Join



Gemini Notebook

O Perigo do Espaço Compartilhado

Por que compartilhar memória é vantajoso e destrutivo ao mesmo tempo?



O Problema: A comunicação entre threads é instantânea via variáveis globais (vantagem). Mas se duas threads lerem e escreverem exatamente no mesmo nanossegundo, a ordem temporal corrompe o dado. Bem-vindo à Condição de Corrida (Race Condition).

Gemini Notebook

Dissecando a Condição de Corrida (Interleaving)

A instrução “contador++” não é atômica. Na CPU, ela esconde 3 passos independentes (Ler, Somar, Gravar).

Tempo (Ciclo CPU)	Thread 1 (Magenta)	Thread 2 (Laranja)	Memória Global
T1	Lê contador (Valor 0)	(em espera)	0
T2	(sofre preempção)	Lê contador (Valor 0)	0
T3	Soma 1 e grava na RAM	(em espera)	1
T4	(concluída)	Soma 1 e grava na RAM	1 (Sobrescreveu a T1!)

Impacto

O **Resultado Imprevisível**: Matematicamente, esperávamos o valor 2. Recebemos 1. A falha de sincronização gerou um resultado não determinístico, corrompendo a lógica do sistema baseada na sorte do escalonamento.



Gemini Notebook

A Cura: Região Crítica, Exclusão Mútua e Mutex

Região Crítica: A porção de código onde manipulamos ativamente os recursos compartilhados.

O Caos (Sem Proteção)

```
contador++; // Colisão iminente!
```

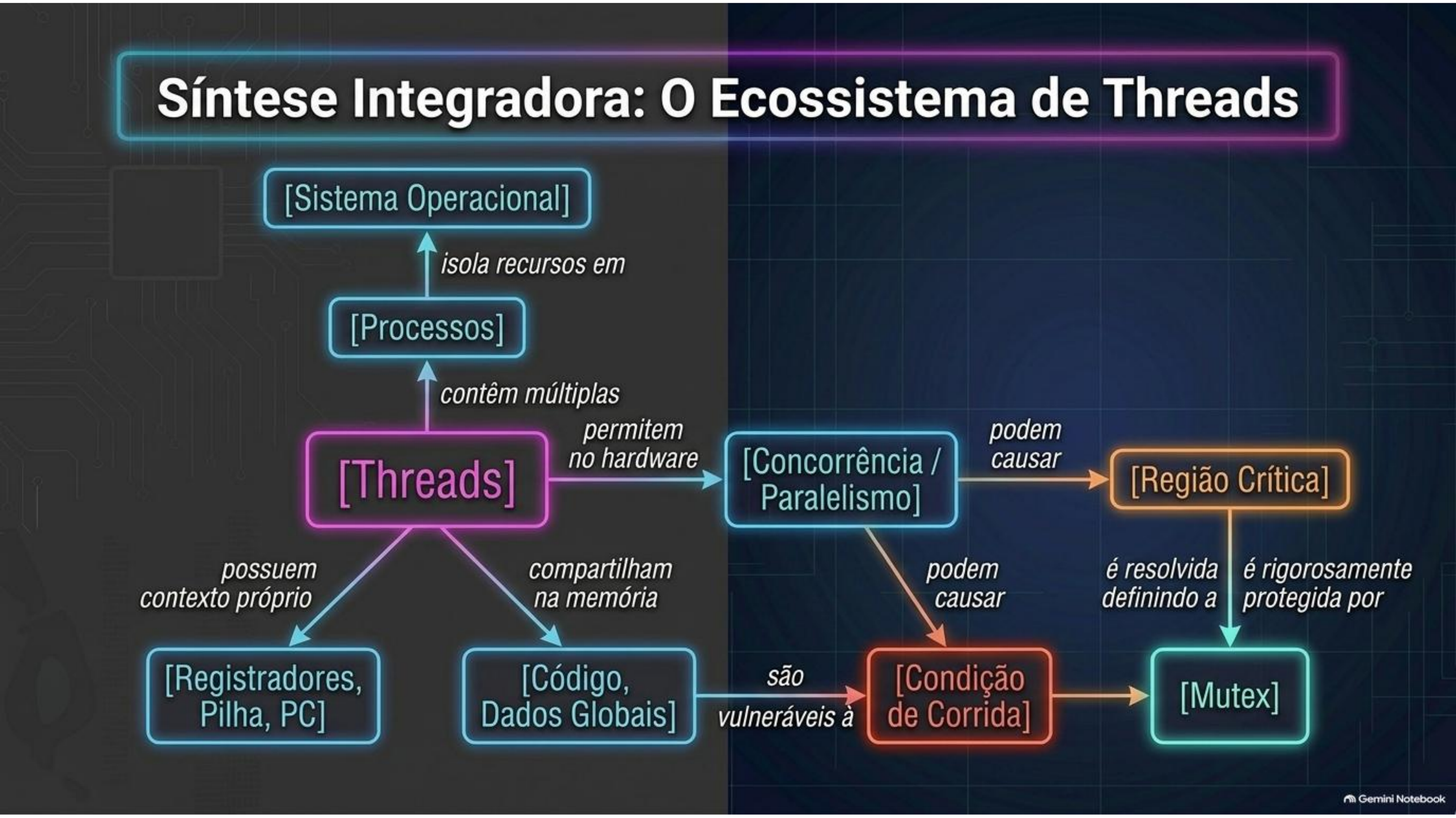
A Ordem (Com Mutex)

```
pthread_mutex_lock(&mutex); // Tranca a porta
contador++;                 // Região Crítica Segura
pthread_mutex_unlock(&mutex); // Abre a porta
```

A Solução: Garantir a Exclusão Mútua. Apenas uma thread pode possuir a trava (mutex) por vez. Se uma segunda thread tentar entrar, o Sistema Operacional a suspenderá até que a porta seja liberada.

Gemini Notebook

Síntese Integradora: O Ecossistema de Threads



Gemini Notebook

O Que Devemos Levar Desta Aula?

1. O Motor da Execução

Threads são os verdadeiros fluxos de trabalho da CPU. São incrivelmente leves e econômicas para criar, mas não sobrevivem independentes do processo-pai.

2. O Meu e o Nosso

Identifique fronteiras: O código, arquivos abertos e dados globais são da família (compartilhados). O histórico de execução (Pilha e Registradores) é estritamente pessoal (privado).

3. A Falsa Simultaneidade

Concorrência não exige múltiplos núcleos; a alternância rápida de contexto engana nossos sentidos. Já o Paralelismo é a verdadeira execução física simultânea no hardware.

4. Responsabilidade do Compartilhamento

Variáveis em comum exigem sincronização matemática. Sem proteções via SO (como Mutex), as condições de corrida corromperão os dados de forma imprevisível.

> exit 0

Gemini Notebook